

Reviewer Pack — Real Rank Lower Bound for AC^0 Communication Matrices of PARI...

Entry 50fc0919c645 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 15:23:55 UTC

Real Rank Lower Bound for AC^0 Communication Matrices of PARITY

Entry ID: 50fc0919c645 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 15:23:55 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For any AC^0 circuit C computing the parity function on n bits, the real rank of the communication matrix of C is $\Omega(n^{\{1/(d-1)\}})$, where d is the depth of C . This implies that the communication complexity of C is $\Omega(n^{\{1/(d-1)\}})$.

Rationale (proposer's reasoning):

The real rank of the communication matrix captures the geometric complexity of the function's representation. High real rank suggests that the function cannot be approximated by low-depth circuits, aligning with known lower bounds from switching lemmas.

Taxonomy category: AC^0_PARITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 627d2dbd34736d5a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_matrix(circuit):
        n = len(circuit)
        comm_matrix = [[0] * (n * n) for _ in range(n * n)]
        for i in range(n):
            for j in range(n):
                if circuit[i][j]:
                    comm_matrix[i * n + k][j * n + k] = 1
        return comm_matrix

    def real_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(min(m, n)):
            if matrix[i][i]:
                rank += 1
                for j in range(i + 1, m):
                    factor = Fraction(matrix[j][i], matrix[i][i])
                    for k in range(n):
                        matrix[j][k] -= factor * matrix[i][k]
        return rank

    def ac0_circuit(n):
        circuit = [[False] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    circuit[i][j] = True
                    circuit[j][i] = True
        return circuit

    n = random.randint(5, 40)
    circuit = ac0_circuit(n)
    comm_matrix = communication_matrix(circuit)
```

```

rank = real_rank(comm_matrix)

expected_rank = math.ceil(n ** (1 / (len(circuit) - 1)))
conjecture_holds = rank >= expected_rank

return {
    "metric_name": "real_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"n={n}, rank={rank}, expected={expected_rank}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={results[0]['metric_value']}, rank={results[0]['real_rank']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9e3ce890.py", line 73, in <module>
 result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9e3ce890.py", line 53, in run_trial

comm_matrix = communication_matrix(circuit)
 ^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9e3ce890.py", line 27, in communication_matrix

comm_matrix[i * n + k][j * n + k] = 1
 ^

NameError: name 'k' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to NameError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the NameError in the communication_matrix function and re-run tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	114548
2	propose	ollama_remote	qwen3:8b	0	0	119443
3	preregistration	ollama_remote	qwen3:8b	0	0	27729
4	novelty	ollama_remote	qwen3:8b	0	0	24408
5	novelty	ollama_remote	qwen3:8b	0	0	18900
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12228
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9479
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12107
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8789
10	judge	ollama_remote	qwen3:8b	0	0	18760

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 366391 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/50fc0919c645.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/50fc0919c645.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/50fc0919c645.tar.gz` (if generated)